

CP020003

ARTIFICIAL INTELLIGENCE



Khon Kaen University · 2026

Week 06

Recommender Systems

From “People Also Bought” to Deep Learning



Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1

What We'll Cover



01

Why recommenders matter

The catalog problem & the family tree of techniques



02

Our dataset

1.1M real book ratings — explore & clean it together



03

Six techniques, live

Popularity → CF → SVD → Content-Based → Deep Learning



04

Evaluate honestly

RMSE, Precision@K, Recall@K, Coverage



05

Modern & production

Two-tower, transformers, GNNs, hybrid pipelines



06

Make it a business

Turning a list of books into real revenue

1 • THE PROBLEM

Why Recommender Systems? 🤔

A bookstore with 10 books on a shelf doesn't need AI — a human can browse all of them.

Our dataset has 271,360 books. No reader will ever browse that shelf.



Search

User intent is explicit — they typed a query.

EXAMPLES

Google Search, site search bars



Recommendation

User intent is implicit — they want something good, and may not know what.

EXAMPLES

Netflix “Because you watched...”, Amazon “Customers also bought”, Spotify Discover Weekly

271,360

books in today's
catalog — far too
many to browse

The whole point of a recommender system: personalized filtering at scale.

The Recommender Systems Family Tree



Popularity-Based

Recommend what's popular to everyone



Content-Based Filtering

“You liked X → here's more like X”, using item features



Collaborative Filtering

“Users like you liked this” — user-based & item-based



Matrix Factorization

Learn hidden latent factors for every user & item



Deep Learning

Learn embeddings with a neural network

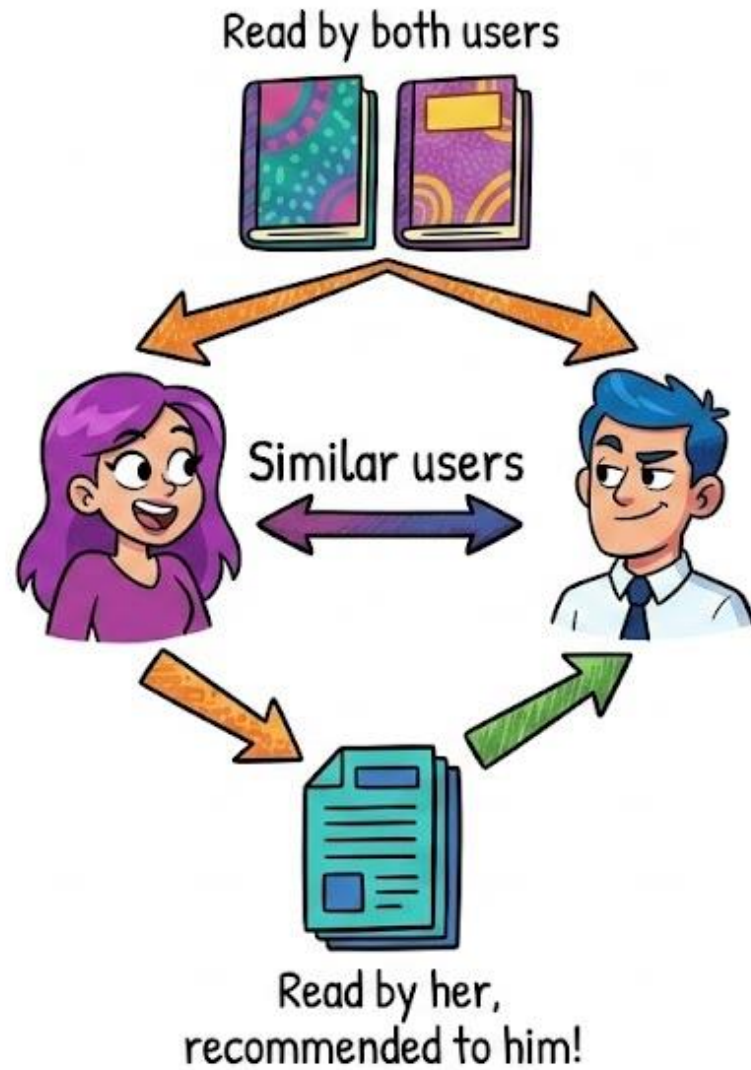


Hybrid

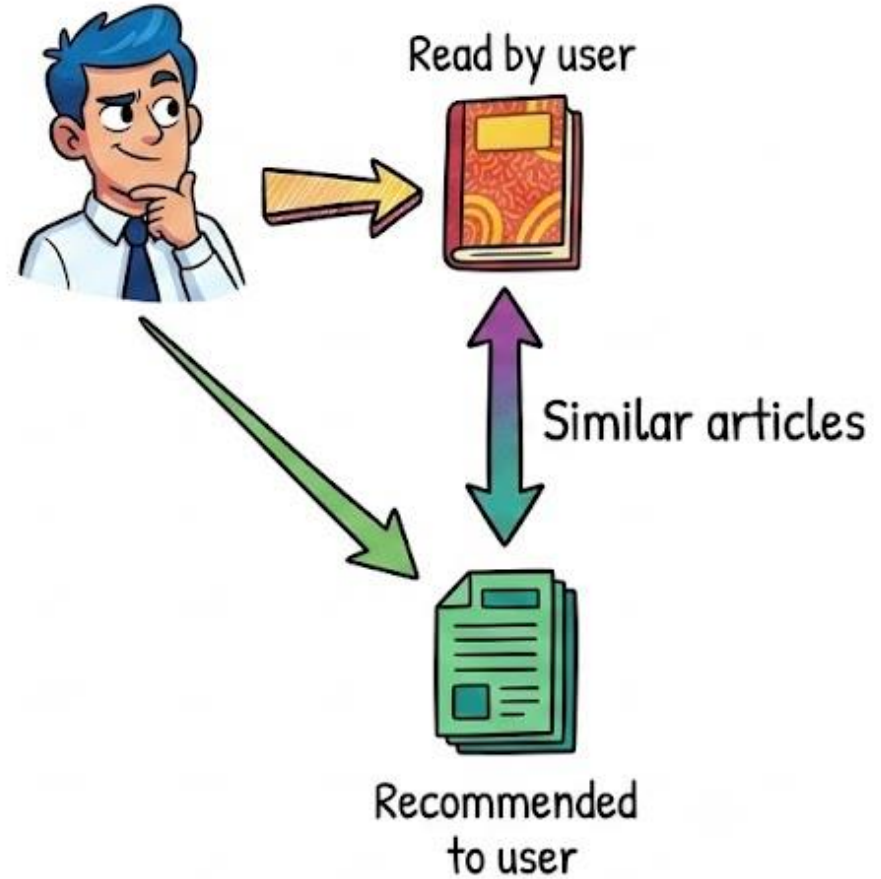
Combine two or more of the above

We'll build one working example of almost every row — on the same dataset.

COLLABORATIVE FILTERING

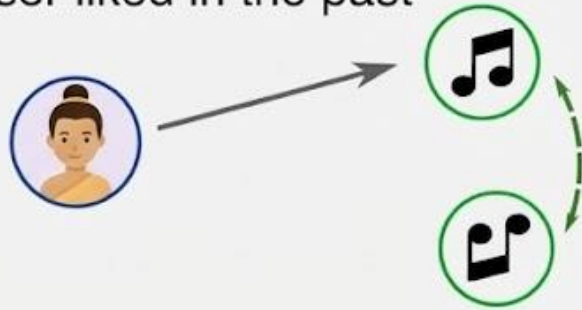


CONTENT-BASED FILTERING



Content-Based

- Use items metadata / tags
- Suggest items similar to what user liked in the past



Recommended



MORE LIKE
MILLI



T-Pop Revolution
Various Artists



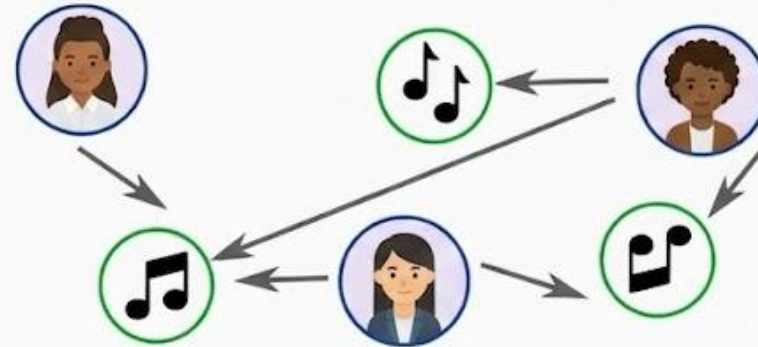
SPRITE x GUYGEEGEE
SPRITE x GUYGEEGEE



The Toys
The Toys

Collaborative Filtering

- Use all feedbacks from all users
- Similar users like similar items



Recommended

Made For You

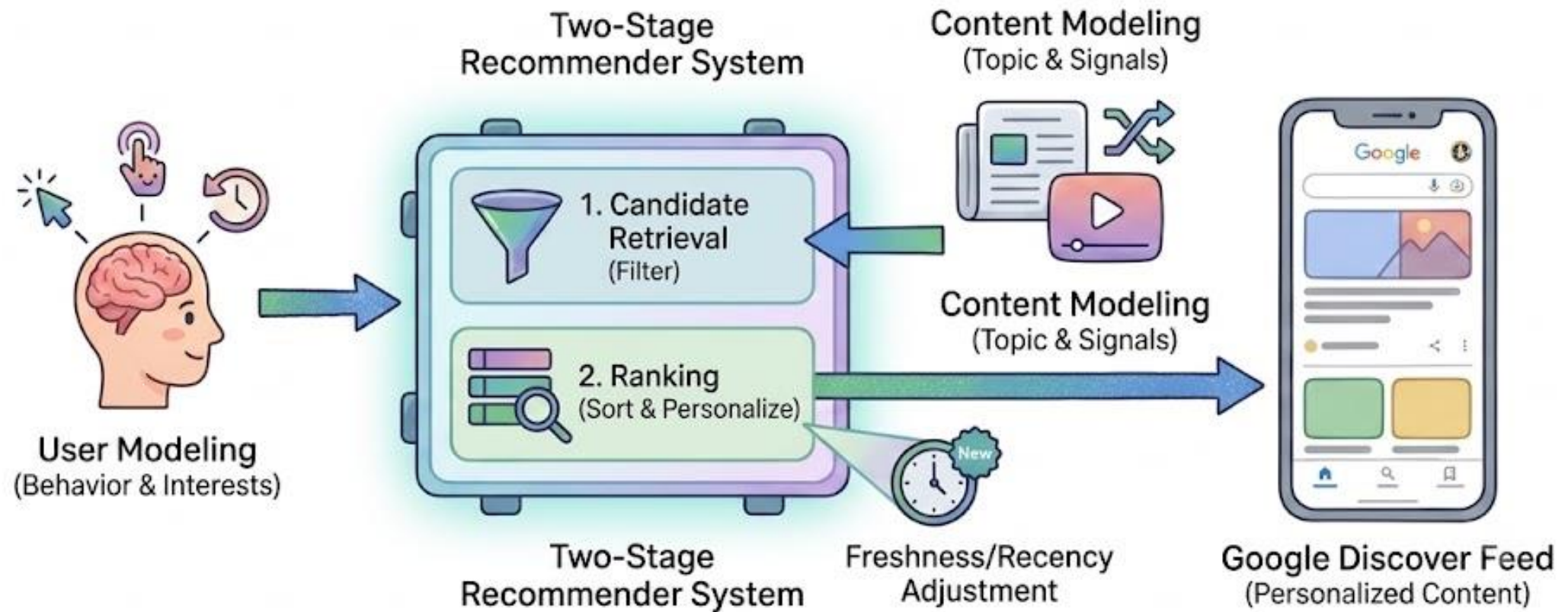


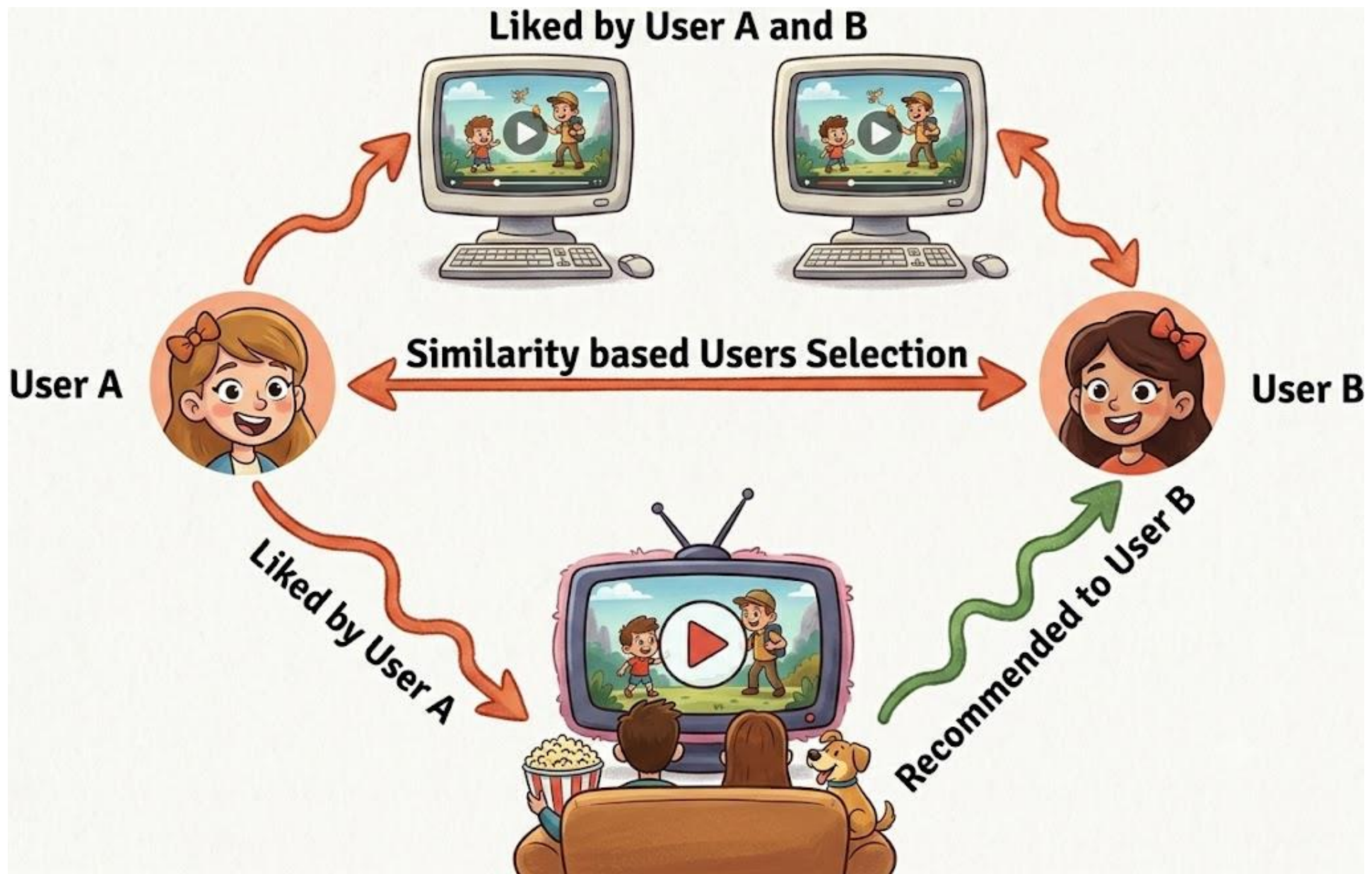
Discover Weekly
Enjoy new T-Pop and
hip-hop chosen just for you!



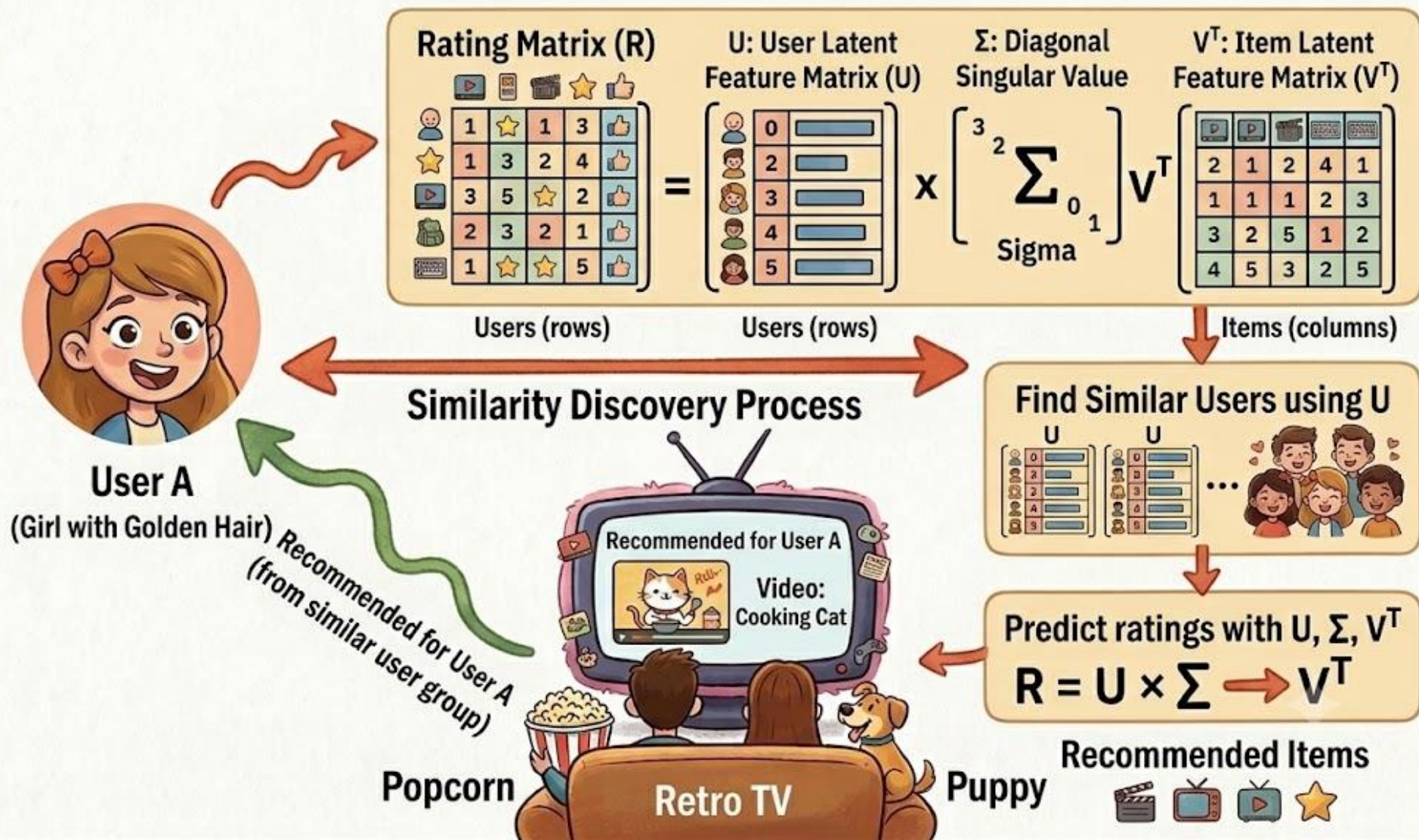
Your Daily Drive
Fans like you also
like these Thai tracks!

Recommender System Design Offers Clues to How Google Discover Surfaces Content

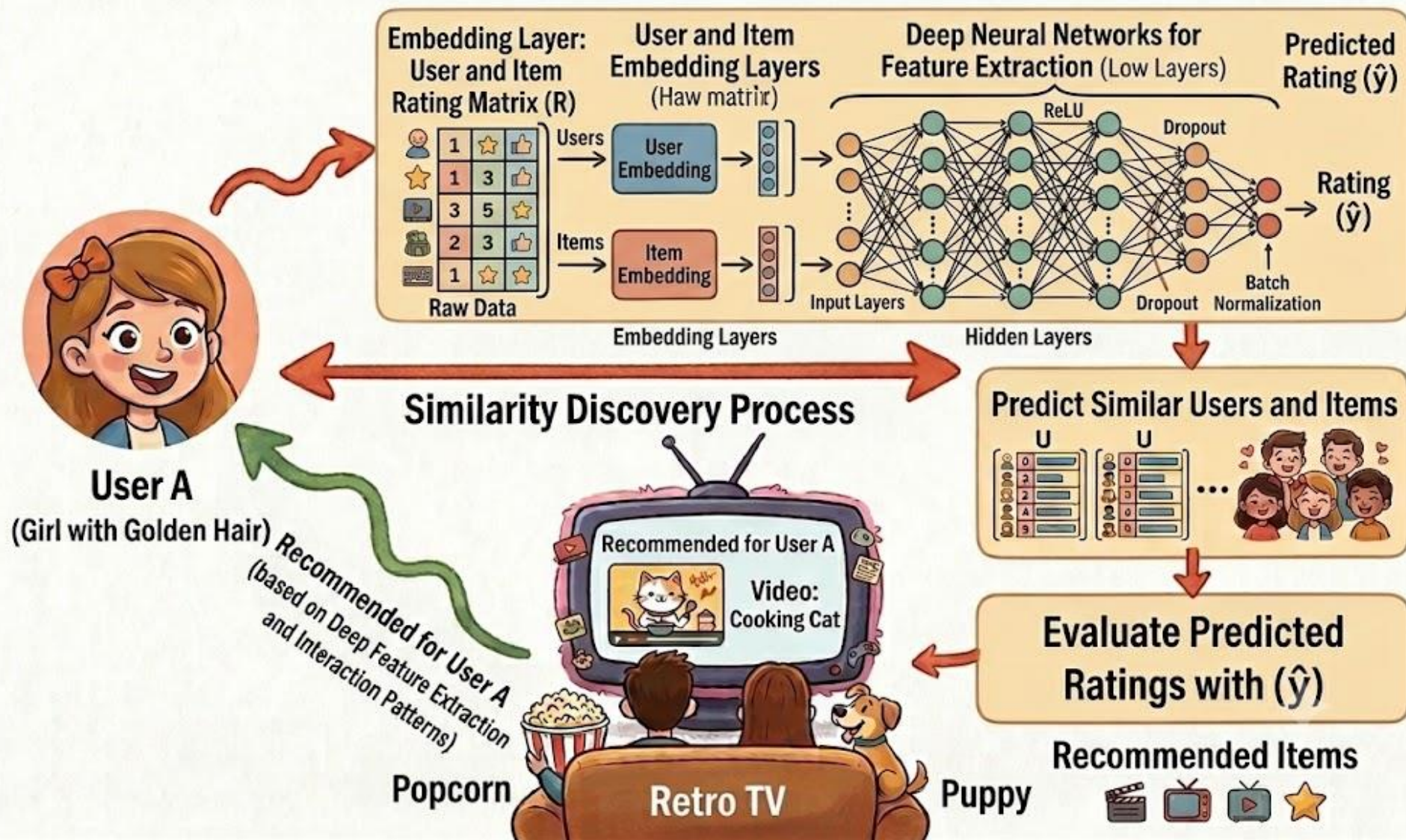




SVD-based Recommendation System (Singular Value Decomposition)



Deep Learning-based Recommendation System (Neural Collaborative Filtering)



3 • THE DATASET

Our Dataset Today: Book-Crossing

Real-world book ratings collected from the Book-Crossing community — exactly the kind of messy, sparse data you'll meet in the real world.



271,360

Books



278,858

Users



1,149,780

Ratings



0-10

Rating scale
(0 = implicit)

Books.csv ISBN, title, author, year, publisher, cover URLs

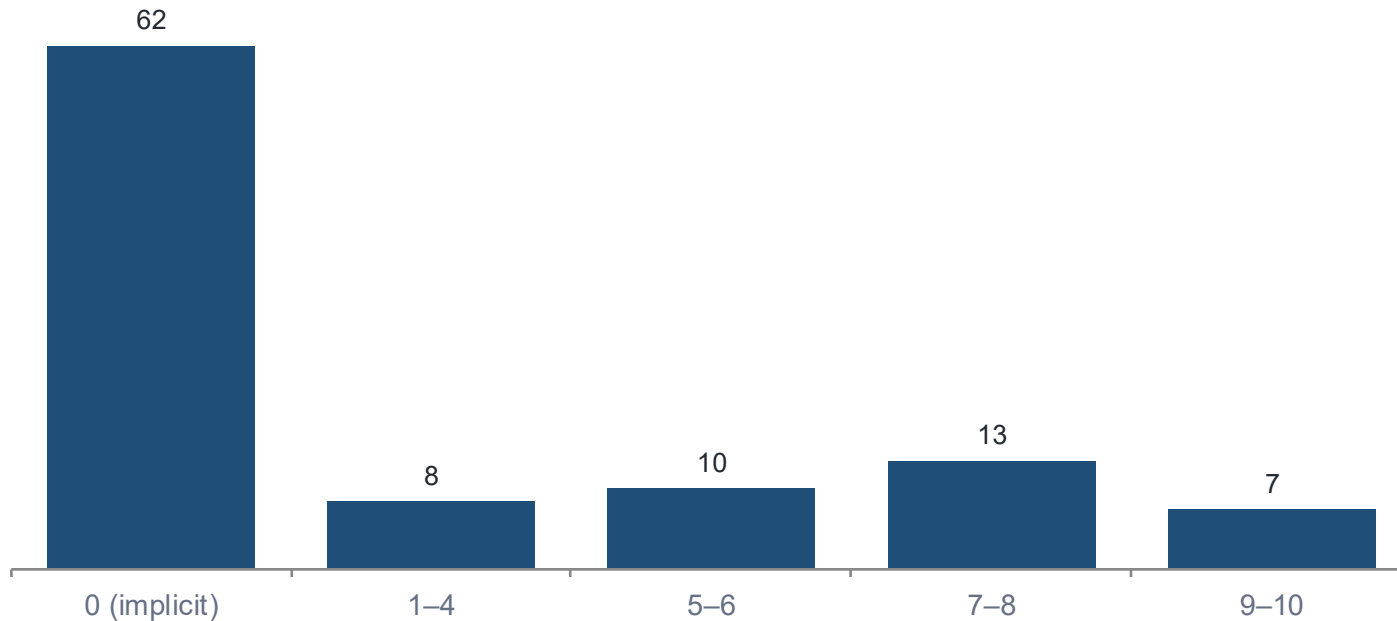
Ratings.csv User-ID, ISBN, Book-Rating (0-10)

Users.csv User-ID, Location, Age

First Look: Most Ratings Are “0”

A rating of 0 means the user interacted with a book but left no star rating — an implicit signal, not “they hated it.”

Book-Rating value distribution (%)



62.3%

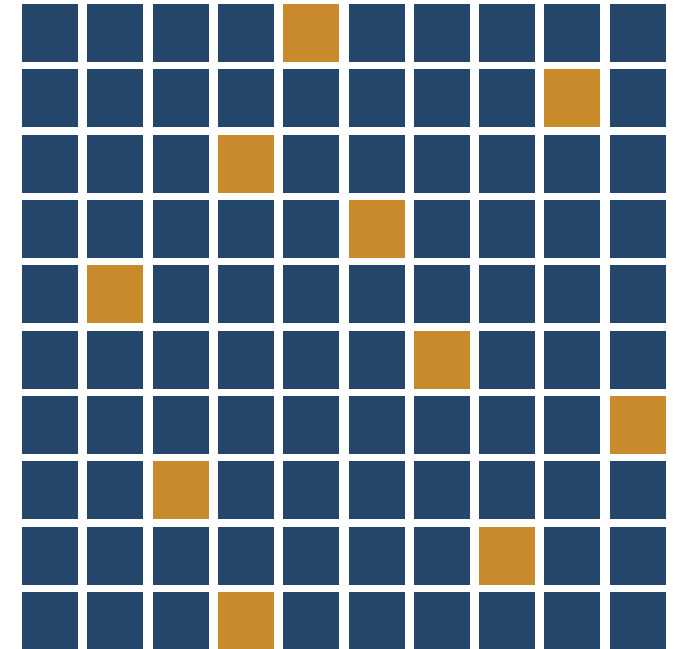
of all interactions
are implicit (no star
rating given)

→ We'll filter to explicit ratings (1–10) only
before modeling.

99.9968%

105,283 users × 340,556 books — but only 1,149,780 ratings exist. The average user has rated a tiny fraction of a percent of the catalog.

 *Trick #1 — sparsity drives most engineering decisions in this field: filtering, embeddings, and approximate search all exist because of this one number.*



Each cell = one (user, book) pair — gold = rated, blue = unknown

Cleaning & Filtering: Before vs. After ✂

1. Keep explicit ratings only (Book-Rating > 0)
2. Filter to active users & popular books (≥ 15 ratings each)

BEFORE

Explicit ratings

433,671

Unique users

77,805

Unique books

185,973



AFTER

Explicit ratings

62,344

Unique users

4,574

Unique books

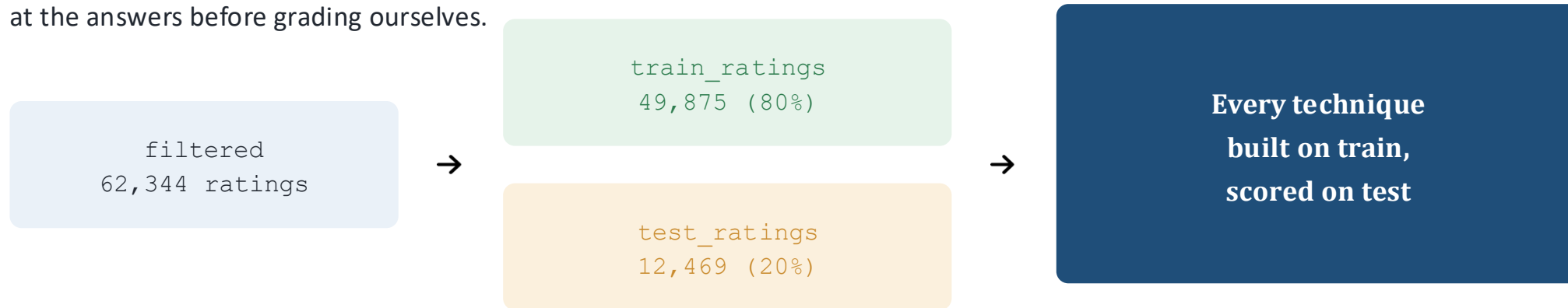
3,149



Smaller, denser, and every user/book now has enough signal to actually learn from.

Golden Rule: One Train/Test Split, Used Everywhere

Every technique today is trained on `train_ratings` and evaluated on the SAME held-out `test_ratings`. This avoids evaluation leakage — “peeking” at the answers before grading ourselves.



This is the exact same discipline used in every serious ML evaluation — not just recommenders.

Popularity-Based Recommender



Recommend whatever is popular to **everyone**. Sounds too simple to matter — but it's the mandatory **cold-start fallback** every real system needs.

We use a Bayesian average (like IMDb's "Top Rated") so one 10/10 rating can't beat 500 ratings averaging 9.2:

```
C = book_stats["n_ratings"].quantile(0.9)
m = train_ratings["Book-Rating"].mean()

book_stats["bayesian_score"] = (
    n / (n + C) * avg_rating + C / (n + C) * m
)

recommend_popular(k=10)
```

TOP RECOMMENDATIONS (EVERYONE)

Harry Potter and the Goblet of Fire

J. K. Rowling

Harry Potter and the Prisoner of Azkaban

J. K. Rowling

The Return of the King

J. R. R. Tolkien

To Kill a Mockingbird

Harper Lee

 **Trick #2** — Popularity is your cold-start fallback for every user with zero history.

User-Based Collaborative Filtering



“Find users whose taste looks like mine, then recommend what *they* liked that I haven't read.” Cosine similarity across the user × book matrix finds the neighborhood.

```
user_similarity = cosine_similarity(
    user_item_matrix.values
)

neighbors = user_sim_df[user_id] \
    .sort_values(ascending=False) \
    .head(20)

recommend_user_based(user_id=242)
```

RECOMMENDED FOR USER 242

Fahrenheit 451

Ray Bradbury

I, Robot

Isaac Asimov

Prodigal Summer: A Novel

Barbara Kingsolver

Life of Pi

Yann Martel

 **Trick #3** — User-based CF doesn't scale. $O(N^2)$ similarity across 280,000 users is expensive — individual taste is also noisy and drifts fast.

Item-Based Collaborative Filtering



Flip it: “find books rated similarly by the same people.” This is Amazon's classic **“Customers who bought this also bought.”** More stable & cheaper to precompute than user-based.

```
item_similarity = cosine_similarity(  
    user_item_matrix.T.values  
)  
  
weights = item_sim_df[rated.index] \  
    .values @ rated.values  
  
recommend_item_based(user_id=242)
```

Because you liked: “The Lovely Bones” →

SIMILAR BOOKS

Lucky: A Memoir

Alice Sebold

Where the Heart Is

Billie Letts

The Lake House

James Patterson

✅ *Why it wins in production: the catalog is far more stable than the user base — item similarity can be precomputed once and cached.*

Matrix Factorization (SVD)

Every user & book gets ~20 hidden latent factors. Multiplying a user's factors by a book's factors predicts the rating — even for books they never rated.

$$R \approx U \cdot V^T$$

```
svd = TruncatedSVD(n_components=20)
user_factors = svd.fit_transform(demeaned)
item_factors = svd.components_.T

predicted = user_factors @ item_factors.T
predicted += user_means # add taste back
```

 This exact idea won the

\$1,000,000 Netflix Prize

in 2009 — the ancestor of every neural embedding technique that followed.

TEST RMSE (1–10 SCALE)

1.61

Lower is better — comparable to deep learning, milliseconds to train

 **Trick #4** — Mean-centering first (subtract each user's average rating) keeps predictions realistic on a matrix that's 99%+ zeros.



Content-Based Filtering

Every technique so far has one blind spot: a brand-new book has **zero ratings** and can never be recommended. Content-based filtering fixes this using the book's own text.

```
tfidf = TfidfVectorizer(  
    stop_words="english",  
    max_features=20000  
)  
text = catalog["Title"] + \  
    " " + catalog["Author"]  
tfidf_matrix = tfidf.fit_transform(text)  
  
recommend_content_based(seed_isbn)
```

Because you liked: "The Lovely Bones" →

TEXTUALLY SIMILAR BOOKS

Lucky: A Memoir

Alice Sebold — same author

The Last Lovely City

Alice Adams

The Almost Moon

Alice Sebold

 **Trick #5** — Combine, don't choose. Use content-based for new items until they gather ~10–20 ratings, then hand off to CF. That's a hybrid.

Deep Learning: Neural Collaborative Filtering



SVD predicts a rating as a fixed dot product. A neural network can learn a more flexible, non-linear function instead — the same idea (scaled up) behind YouTube and TikTok's feeds.

```
user_emb = Embedding(n_users, 16)(user_in)
item_emb = Embedding(n_items, 16)(item_in)

x = Concatenate()([user_vec, item_vec])
x = Dense(32, activation="relu")(x)
x = Dense(16, activation="relu")(x)
output = Dense(1)(x)

model.fit([train.u, train.i], train.rating)
```

RECOMMENDED FOR USER 242

The Little Prince

Antoine de Saint-Exupéry

Pride and Prejudice

Jane Austen

CONTACT

Carl Sagan

1.57

Test RMSE

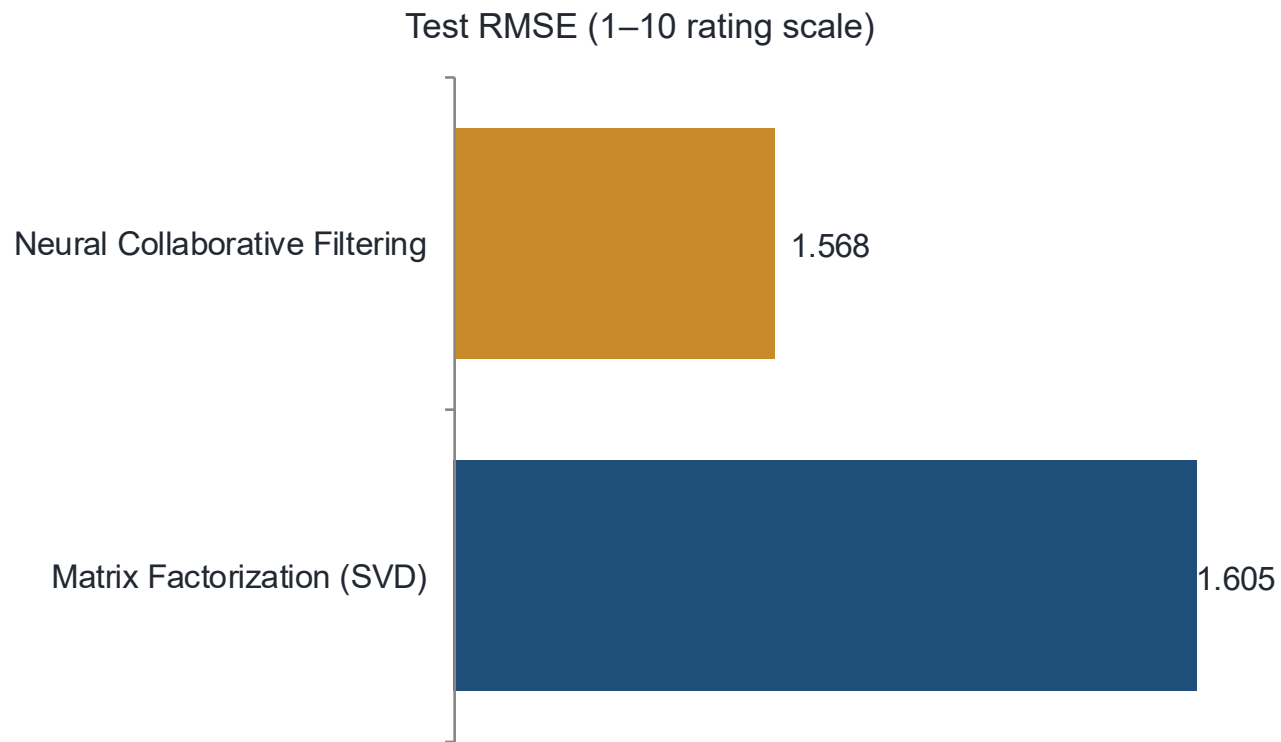
8 epochs

CPU, < 1 min

 **Trick #6** — Deep learning isn't automatically "better." With ~60K interactions, SVD and Neural CF land in the same ballpark — always try the simple baseline first.

SVD vs. Neural CF: Head-to-Head

Both predict an actual rating value, so we can score both with the same metric on the same held-out test set.



	SVD	Neural CF
Training time	Milliseconds	Seconds (CPU)
Handles new items?	No	No
Handles new users?	No	No

Takeaway: try the cheap linear-algebra baseline first — SVD gets 97% of Neural CF's accuracy at a fraction of the training cost.

Evaluation Beyond RMSE

Nobody shows a user a predicted number. They see a ranked LIST. What matters is: of what we recommended, how many did they actually want?



Precision@K

Of the K items we recommended, what fraction did the user actually like?



Recall@K

Of everything the user actually liked, what fraction did we surface in our top K?



Coverage

What fraction of the ENTIRE catalog does the system ever recommend to anyone?

 *Trick #7 — Never evaluate on accuracy alone. Always-recommend-the-#1-bestseller can score well on precision while giving zero real personalization.*

Popularity vs. Item-Based CF: The Real Numbers

Precision@10

Popularity

0.53%

Item-Based CF

1.87%

Item-based CF: 3–4× better

Recall@10

Popularity

1.93%

Item-Based CF

7.18%

Item-based CF: 3–4× better

Coverage

Popularity

0.32%

Item-Based CF

20.4%

Item-based CF: 64× more of the catalog reachable

Popularity often looks fine on precision — but its coverage is tiny: it recommends the same handful of bestsellers to everyone. Item-based CF trades a little precision for real personalization.

A Guided Tour of Modern Techniques



Two-Tower Retrieval

Embed users & items in the same space; search millions of items in milliseconds

YouTube candidate generation



Sequential / Session-Based

Model “what's next” from the last 10 clicks using Transformers

TikTok's For You feed



Graph Neural Networks

Propagate signal across a user–item graph, multiple hops out

Pinterest (PinSage)



LLM-Based / Conversational

Reason over item text in natural language; users describe what they want

Conversational shopping



Hybrid Multi-Stage

Cheap retrieval narrows millions → hundreds, then a heavy ranker re-scores

Virtually every large platform

The pattern to remember: fast retrieval (get to hundreds) → slower, precise ranking (pick the best few).

Tricks & Pitfalls Every Practitioner Should Know



Cold Start (users & items)

Fall back to popularity, or use content-based / onboarding questions



Popularity Bias

Over-recommending bestsellers reinforces itself — track coverage, not just accuracy



Implicit vs. Explicit Feedback

A click is not a 1–10 rating — conflating them corrupts the model



Negative Sampling

Datasets record what users liked; sample non-interactions as pseudo-negatives



Evaluation Leakage

Never let a user's future rating leak into training — split first, always



Scalability at Inference

Brute-force similarity is too slow at scale — use approximate nearest neighbors

Turning Recommendations Into a Business Action

Segment	Technique	Action
Brand-new visitor	Popularity-Based	“Bestsellers” homepage rail — zero personalization needed
New signed-up user	Content-Based	Ask 3 favorite genres at signup, recommend immediately
Established reader	Item-Based CF / SVD / NCF	Personalized “Recommended for you” email
Lapsed 60+ days	Item-Based CF (seeded)	“We picked these because you loved ...” win-back email

WORKED EXAMPLE

4,498 active users × 8% conversion × \$15 avg. price

\$5,398

incremental revenue
from ONE personalized email send

That's the entire economic case for a recommender in one line: turning a static catalog into a personalized nudge, multiplied across your user base.

Key Takeaways



Sparsity drives everything

99.99%+ of a rating matrix is empty — that single fact shapes the whole field



Item-based CF usually wins over user-based

Item similarity is stable & cheap to precompute



Content-based fixes item cold start

The one weak spot every collaborative method shares



Evaluate with Precision/Recall/Coverage

Accuracy alone hides a system with zero real personalization



Popularity is not a toy

It's the mandatory cold-start fallback in every real system



SVD compresses sparsity into taste

The Netflix Prize technique — ancestor of neural embeddings



Deep learning isn't automatically better

Try the cheap linear baseline first — then justify the upgrade



Connect it to dollars

A recommendation list only matters once it drives a business action

This Week's Homework



A new dataset — with a twist

The e-commerce dataset gives you a customer table and a product table — but no shared rating column linking them.

- 1 Engineer your own implicit customer–product interaction signal
- 2 Select ≥ 2 techniques from today's class
- 3 Train, generate recommendations, and evaluate honestly on held-out data
- 4 Compare your techniques head-to-head, then recommend one per customer segment



Special Recognition

+1% extra credit

for the clearest technique comparison, backed by real Precision@K / Recall@K / Coverage numbers, with a well-argued deployment recommendation per segment.



Questions?

Thank you — let's build some recommenders. 📖